Roteiro de demonstração - Conversor distribuído

IFPE | Sistemas Distribuídos | Áudio e vídeo | Gravação de até 10 minutos

1. Preparação e tempo da gravação

Faça um ensaio antes de gravar para escolher vídeos que convertam em um tempo adequado à sua máquina. Procure terminar até 9min30s, deixando margem para o limite de dez minutos. Use arquivos de teste e um e-mail local, como demo@conversor.local.

Tempo	O que mostrar
0:00-0:40	Objetivo, arquitetura e dois workers disponíveis.
0:40-2:00	Upload pela interface, aceite com jobId e pedidos visíveis na fila.
2:00-3:10	Dois workers processando jobs distintos e resultado de uma conversão.
3:10-5:50	Novo job, interrupção do worker ativo e espera/recuperação por outra réplica.
5:50-7:30	CONCLUÍDO, resultado no MinIO, origem removida, e-mail e arquivo aberto.
7:30-8:50	Comparação medida com um e dois workers e evidências de testes.
8:50-9:30	Garantias, limites e encerramento.

Antes de começar a gravar

Na raiz do repositório, com Docker Desktop ativo:

```
docker compose up -d --build --scale conversion-worker=2
docker compose ps
docker compose exec -T api npm run db:check
docker compose exec -T api npm run broker:check
docker compose exec -T api npm run storage:check
docker compose exec -T api npm run smtp:check
```

Confira se os serviços estão funcionando e se há dois workers de conversão, os processos que transformam os arquivos. Separe um áudio, três vídeos para mostrar a fila e um vídeo para o teste de queda. Use arquivos que demorem o suficiente para observar a conversão, mas que terminem dentro do vídeo. O limite por envio é de 100 MiB.

Deixe abertas a página do sistema (localhost:3000), o painel RabbitMQ (localhost:15672), o MinIO (localhost:9001), o Mailpit (localhost:8025) e dois terminais PowerShell. Entre nos painéis antes de gravar. Use um terminal para os comandos e outro para observar as conversões.

Fala de abertura: “Vou mostrar um sistema que converte áudio e vídeo. A página recebe o arquivo e devolve um código para acompanhar o pedido. As conversões acontecem em processos separados. O RabbitMQ organiza a fila, o MinIO guarda os arquivos e o Mailpit permite conferir os e-mails.”

2. Envio, fila e conversões ao mesmo tempo

Mostrar o pedido e a fila

Para conseguir mostrar os pedidos na fila antes de começarem a ser atendidos, pare por um momento apenas os workers de conversão:

```
docker compose stop conversion-worker
```

Na página do sistema, envie um vídeo para WebM e informe demo@conversor.local. Mostre a confirmação e copie o jobId, o código desse pedido. Abra o acompanhamento: ele deve ficar PENDENTE enquanto os workers estiverem parados. Se quiser mostrar a resposta da API, abra a aba Network do navegador e encontre POST /jobs: o código 202 indica que o pedido foi aceito.

Envie mais dois pedidos pela página. No RabbitMQ, abra Queues and Streams e escolha conversion.jobs. Aguarde os eventos chegarem à fila. Mostre Ready maior que zero: são pedidos esperando atendimento. Como os workers estão parados, ainda não haverá consumidores de conversão.

Para olhar o conteúdo de uma mensagem, use Get messages com **Ack mode = Nack message requeue true**, que devolve a mensagem à fila. Mostre os campos eventId, type, jobId, sourceUrl, sourceType, sourceFormat, targetFormat, notifyEmail, requestedAt e attempt. Explique que eles identificam o pedido, o arquivo, o destino e o e-mail. Não escolha a opção que remove a mensagem, nem mostre a assinatura completa do link.

Fala: "O arquivo fica no MinIO. A fila recebe só os dados do pedido e o endereço do arquivo. O banco salva o pedido e uma lista de eventos, chamada outbox. Outro processo envia esses eventos à fila."

Dois workers em paralelo

```
docker compose start conversion-worker
docker compose ps conversion-worker
$replicas = @(docker compose ps -q conversion-worker)
foreach ($replica in $replicas) {
    Write-Output "Worker: $replica"
    docker top $replica -eo pid,args
}
```

Observe os processos assim que os workers começarem. Mostre um FFmpeg em cada container, com caminhos conversion-<jobId>- de pedidos diferentes. No RabbitMQ, mostre dois consumidores e, se aparecer, Unacked=2: duas mensagens entregues que ainda não foram confirmadas. Os dois FFmpegs com pedidos distintos mostram que há conversões acontecendo ao mesmo tempo.

Acompanhe um pedido até CONCLUÍDO e mostre o link para baixar o resultado. A página atualiza a cada três segundos, então uma conversão rápida pode terminar antes de mostrar PROCESSANDO. Se os FFmpegs também terminarem antes da observação, use arquivos um pouco maiores no ensaio e repita com novos pedidos.

Fala: "Cada worker recebe uma mensagem por vez, por causa do prefetch configurado. O banco faz uma reserva temporária para evitar que dois workers assumam o mesmo pedido. Se um resultado já foi salvo, ele pode ser aproveitado na recuperação."

Antes do teste de queda, espere os três pedidos terminarem e confira se não há trabalho esperando nas filas principal e de novas tentativas. Assim, os vídeos anteriores não atrasam a recuperação do próximo pedido.

3. Interromper uma conversão e acompanhar a recuperação

Pare os workers novamente com `docker compose stop conversion-worker`. Envie o vídeo separado para o teste de queda e copie seu jobId. Confira PENDENTE na página e Ready na fila. Execute o bloco abaixo e informe esse código. Ele encontra o FFmpeg do pedido, mostra em qual container está rodando e interrompe esse container.

```
$jobAlvo = Read-Host "Cole o jobId do teste de queda"
$replicaInterrompida = $null
docker compose start conversion-worker
$replicas = @(docker compose ps -q conversion-worker)
if ($replicas.Count -ne 2) { throw "Esperadas duas replicas" }
$limite = (Get-Date).AddSeconds(30)
while ((Get-Date) -lt $limite -and -not $replicaInterrompida) {
    foreach ($replica in $replicas) {
        $processos = docker top $replica -eo pid,args
        $ffmpeg = $processos | Where-Object {
            $_ -match "ffmpeg" -and $_.Contains("conversion-$jobAlvo-")
        }
        if ($ffmpeg) {
            Write-Output "Interrompendo: $replica"
            $ffmpeg
            $replicaInterrompida = $replica
            docker stop -t 0 $replicaInterrompida
            break
        }
    }
    if (-not $replicaInterrompida) { Start-Sleep -Milliseconds 200 }
}
if (-not $replicaInterrompida) {
    throw "FFmpeg do job nao encontrado; repita com arquivo maior"
}
docker compose ps -a conversion-worker
```

A interrupção precisa acontecer enquanto o FFmpeg está convertendo o pedido. Confira o jobId e o container mostrados pelo comando. docker stop -t 0 faz uma parada imediata, sem apagar os dados armazenados. Deixe o outro worker funcionando.

Durante a espera

Mostre PROCESSANDO na página e acompanhe a fila. Como o worker caiu sem confirmar a mensagem, ela pode voltar ao RabbitMQ. Porém, o banco mantém uma reserva de 120 segundos para aquele pedido. O outro worker precisa esperar essa reserva vencer. Se receber o pedido enquanto ele ainda está reservado, espera mais 30 segundos para tentar novamente; por isso, a retomada pode levar um pouco mais de dois minutos. Use o outro terminal para acompanhar:

```
docker compose exec -T rabbitmq rabbitmqctl list_queues -p / `
    name messages_ready messages_unacknowledged consumers
docker compose logs --tail=30 conversion-worker
```

Durante a espera, use o diagrama do relatório para explicar: o ack confirma que a mensagem foi tratada; o lease é a reserva temporária do pedido; e os retries são novas tentativas. Se encurtar essa espera na edição, informe quanto tempo passou e mantenha as imagens de antes e depois.

Mostre o FFmpeg do mesmo jobId no **container que continuou funcionando**, usando docker top. Depois, aguarde CONCLUÍDO. Isso mostra que o pedido foi retomado pelo outro worker, além de apenas exibir o resultado final.

4. Conferir o resultado, o e-mail e os testes

Conferir o ciclo completo

No MinIO, procure pelo jobId do pedido recuperado. Em converted, mostre <jobId>/result.<formato>. Em uploads, confira que o original desse pedido foi removido. Se ele ainda estiver lá, aguarde e atualize a tela: a limpeza acontece depois de registrar CONCLUÍDO. Confira sempre o mesmo jobId.

No Mailpit, encontre o e-mail pelo destinatário e pelo jobId no texto. Abra o link e mostre o arquivo convertido funcionando. Um WebM pode abrir no navegador; se for baixado, use um programa compatível para reproduzi-lo. Confira que há um e-mail desse pedido, sem confundi-lo com mensagens de testes anteriores.

Fala: "Primeiro o sistema salva o resultado e confirma a conclusão no banco. Só depois apaga o original. Se essa limpeza falhar, ele tenta apenas a remoção novamente. No e-mail, quando não dá para saber se houve envio, o sistema pede revisão em vez de arriscar mandar outra mensagem."

Comparar um e dois workers

Faça a comparação de workers durante o ensaio, no computador que controla o Docker, com Node.js instalado. Guarde a saída do comando. Não rode junto do teste de queda, porque ele altera a quantidade de workers. Na gravação, você pode executar a comparação se houver tempo ou apresentar a saída do teste já realizado.

```
node scripts/benchmark-workers.mjs
```

O script compara dois lotes de oito áudios WAV de 30 segundos: um lote com um worker e outro com dois. Ele mede do envio até o fim das conversões e, depois, devolve o ambiente à quantidade inicial de workers. Mostre os números da execução que está apresentando. Em output/validation/results.json, a medição registrada foi de 3,290 segundos e 2,432 arquivos por segundo com um worker; com dois, 1,969 segundos e 4,064 arquivos por segundo. Isso representa cerca de 67,1% a mais de arquivos concluídos por segundo.

Mostre também output/validation/correcoes-resultados.json. Ele registra 60 testes aprovados no Windows e no Linux, a verificação de tipos do TypeScript e 21 cenários com os serviços integrados. Deixe claro que são resultados dos testes anteriores. Explique que uma conversão tem até três tentativas, com esperas de 5 e 30 segundos. As mensagens que falham de forma definitiva vão para a DLQ, a fila de erros. Se não for possível saber se o e-mail foi aceito, ele pode ficar em FAILED antes da terceira tentativa e precisa de revisão.

5. Encerrar e preparar o ambiente para o próximo uso

Fala final: “Mostramos o envio pela página, a fila, duas conversões ao mesmo tempo e a recuperação de um pedido depois de interromper um worker. Também vimos o arquivo convertido, a remoção do original e o e-mail com o link. O sistema foi preparado para uso local. Ele permite recuperar pedidos, mas isso não significa que toda operação aconteça exatamente uma vez.”

Depois de gravar, volte à quantidade de workers que usava antes. Neste ambiente, era um worker. Se você começou com outra quantidade, ajuste o número no comando:

```
docker compose up -d --scale conversion-worker=1 conversion-worker
docker compose ps
```

Mantenha os pedidos, e-mails, arquivos e mensagens de erro usados na demonstração. Não é necessário apagar volumes ou esvaziar filas. Antes de entregar o vídeo, confira: até dez minutos, códigos dos pedidos legíveis, dois FFmpegs com pedidos diferentes, interrupção durante uma conversão, retomada pelo outro worker, original removido, resultado salvo e arquivo aberto pelo link do e-mail.

Arquivo de teste opcional, antes do ensaio

Se precisar de um arquivo de teste, gere um MP4 pelo FFmpeg que já está no container. Use um worker ativo e ajuste os 45 segundos depois do ensaio. A duração do vídeo não é igual ao tempo que ele leva para ser convertido.

```
New-Item -ItemType Directory -Force -Path tmp/demo | Out-Null
docker compose exec -T conversion-worker ffmpeg -y -f lavfi `
    -i testsrc2=size=640x360:rate=30 -t 45 -c:v libx264 `
    -preset ultrafast /tmp/demo-video.mp4
docker compose cp conversion-worker:/tmp/demo-video.mp4 tmp/demo/video.mp4
```